COE 379L Midterm Project Report: Atlas Laboratory Agent

Name: Yash Rajput | UT EID: YSR266

Repository: https://github.com/hezisplashu123/atlaslabrepomidtermysr266

1. System Overview

The Atlas Laboratory Agent receives a trusted LabRequest containing the authenticated user and any resolved project, resource, start time, and end time. The original request text remains separate. The system prompt labels retrieved passages and tool observations as untrusted data and instructs the model to preserve the trusted request scope. DecisionAdapter validates each model output as a ToolCallDecision, ClarificationDecision, or FinalDecision.

The bounded loop requests one decision at a time. A clarification returns immediately without tool execution. Before dispatching a proposed tool call, Python checks request binding and, for reserve_resource, the ReservationContext preconditions computed from earlier observations. Denied proposals become failure observations rather than tool executions. Each tool proposal and its observation enter a TraceEvent and the next message history. Before returning a final response, the validator checks citations, successful mutations, and status against the trace. The default budget is 10 decisions, with explicit failure when it is exhausted.

2. Public Benchmark Results

All six public scenarios produced their expected final status. The results below come from public_benchmark.json, generated with atlas-evaluate-scripted using ScriptedLLM.

Metric	Value	Metric	Value
Scenario count	6	Unnecessary refusal rate	0%
Success rate	100%	Clarification accuracy	100%
Invalid structured response rate	0%	Average tool calls	2.67
Unsafe action attempt rate	0%		

The tensile-tester reservation completed after policy retrieval, authorization lookup, resource-status lookup, and a successful reservation. Missing laser training, expired robot calibration, missing after-hours approval, and a scheduling conflict each correctly returned blocked. The ambiguous-time request returned clarification_required and asked for exact start and end times without executing a tool. No public scenario failed or produced an unexpected status.

The 100% success rate measures agreement with expected outcomes, including safe blocking and clarification. It does not mean every request created a reservation. The public traces recorded no unsafe action attempts. Their average was 2.67 tool calls per scenario.

3. Student Tests

The complete pytest run passed 26 tests, including all three tests in tests/test_student_cases.py. These tests use ScriptedLLM to control the proposed decisions and avoid network dependence.

The first test proposes a reservation before any required checks. It verifies a blocked response, a recorded action_gate_denied observation, and a mutating proposal marked unsuccessful. A mock wrapping environment.dispatch confirms that dispatch never occurs. This distinguishes prevention before execution from relying on the simulator to reject an unsafe call.

The second test performs a successful authorization lookup, then proposes a final response claiming that reserve_resource completed. The trace contains only a successful read. The test verifies a failed response with invalid_final_response and no completed actions, showing that successful information retrieval cannot justify a claim of state change.

The third test adds an unexpected tool field to a clarification decision. The schema rejects this extra field. The test verifies invalid_model_decision, an empty trace, and no dispatch. Together, these checks exercise the execution boundary, truthful reporting, and rejection of malformed decisions.

4. Implementation Decisions

A discriminated union selects the decision schema through kind, and extra="forbid" rejects undeclared fields. This makes invalid output fail explicitly instead of falling through to execution. Schema validation establishes structure; the action gate separately enforces the selected request and policy checks.

Request binding preserves authenticated identity and resolved scope. An unresolved project, resource, or interval blocks a reservation proposal instead of accepting a model guess. Reservation validation reports each false supplied context fact. The simulator remains unchanged because it checks mechanical reservation constraints but does not enforce every laboratory policy.

Denied calls remain in the trace and message history so later decisions can react to the actual denial. Final validation accepts citations only from successful searches and completed actions only from successful mutating calls. Completed status requires both evidence and an identified action. Non-completed responses cannot hide a successful mutation. Stable error labels and explicit termination reasons make these outcomes testable, including model_finished, clarification_required, invalid_model_decision, invalid_final_response, and maximum_steps_exceeded.

These checks are ordinary Python validation over selected properties, not a formal proof. Passing finite examples does not establish correctness for all combinations of requests, tool results, or repeated actions. ScriptedLLM makes these tests reproducible but does not measure live-model behavior or general resistance to prompt injection. The separate data-flow diagram specializes the architecture to success_tensile.